

BOOT MANAGER

The UEFI boot manager is a firmware policy engine that can be configured by modifying architecturally defined global NVRAM variables. The boot manager will attempt to load UEFI drivers and UEFI applications (including UEFI OS boot loaders) in an order defined by the global NVRAM variables. The platform firmware must use the boot order specified in the global NVRAM variables for normal boot. The platform firmware may add extra boot options or remove invalid boot options from the boot order list.

The platform firmware may also implement value added features in the boot manager if an exceptional condition is discovered in the firmware boot process. One example of a value added feature would be not loading a UEFI driver if booting failed the first time the driver was loaded. Another example would be booting to an OEM-defined diagnostic environment if a critical error was discovered in the boot process.

The boot sequence for UEFI consists of the following:

- The boot order list is read from a globally defined NVRAM variable. Modifications to this variable are only guaranteed to take effect after the next platform reset. The boot order list defines a list of NVRAM variables that contain information about what is to be booted. Each NVRAM variable defines a name for the boot option that can be displayed to a user.
- The variable also contains a pointer to the hardware device and to a file on that hardware device that contains the UEFI image to be loaded.
- The variable might also contain paths to the OS partition and directory along with other configuration specific directories.

The NVRAM can also contain load options that are passed directly to the UEFI image. The platform firmware has no knowledge of what is contained in the load options. The load options are set by higher level software when it writes to a global NVRAM variable to set the platform firmware boot policy. This information could be used to define the location of the OS kernel if it was different than the location of the UEFI OS loader.

3.1 Firmware Boot Manager

The boot manager is a component in firmware conforming to this specification that determines which drivers and applications should be explicitly loaded and when. Once compliant firmware is initialized, it passes control to the boot manager. The boot manager is then responsible for determining what to load and any interactions with the user that may be required to make such a decision.

The actions taken by the boot manager depend upon the system type and the policies set by the system designer. For systems that allow the installation of new Boot Variables (See *Boot Option Recovery*), the Boot Manager must automatically or upon the request of the loaded item, initialize at least one system console, as well as perform all required initialization of the device indicated within the primary boot target. For such systems, the Boot Manager is also required to honor the priorities set in BootOrder variable.

In particular, likely implementation options might include any console interface concerning boot, integrated platform management of boot selections, and possible knowledge of other internal applications or recovery drivers that may be integrated into the system through the boot manager.

3.1.1 Boot Manager Programming

Programmatic interaction with the boot manager is accomplished through globally defined variables. On initialization the boot manager reads the values which comprise all of the published load options among the UEFI environment variables. By using the *SetVariable()* function the data that contain these environment variables can be modified. Such modifications are guaranteed to take effect after the next system boot commences. However, boot manager implementations may choose to improve on this guarantee and have changes take immediate effect for all subsequent accesses to the variables that affect boot manager behavior without requiring any form of system reset.

Each load option entry resides in a *Boot####*, *Driver####*, *SysPrep####*, *OsRecovery####* or *PlatformRecovery####* variable where *####* is replaced by a unique option number in printable hexadecimal representation using the digits 0-9, and the upper case versions of the characters A-F (0000-FFFF).

The *####* must always be four digits, so small numbers must use leading zeros. The load options are then logically ordered by an array of option numbers listed in the desired order. There are two such option ordering lists when booting normally. The first is *DriverOrder* that orders the *Driver####* load option variables into their load order. The second is *BootOrder* that orders the *Boot####* load options variables into their load order.

For example, to add a new boot option, a new *Boot####* variable would be added. Then the option number of the new *Boot####* variable would be added to the *BootOrder* ordered list and the *BootOrder* variable would be rewritten. To change boot option on an existing *Boot####*, only the *Boot####* variable would need to be rewritten. A similar operation would be done to add, remove, or modify the driver load list.

If the boot via *Boot####* returns with a status of *EFI_SUCCESS*, platform firmware supports boot manager menu, and if firmware is configured to boot in an interactive mode, the boot manager will stop processing the *BootOrder* variable and present a boot manager menu to the user. If any of the above-mentioned conditions is not satisfied, the next *Boot####* in the *BootOrder* variable will be tried until all possibilities are exhausted. In this case, boot option recovery must be performed (See *Boot Option Recovery*).

The boot manager may perform automatic maintenance of the database variables. For example, it may remove un referenced load option variables or any load option variables that cannot be parsed, and it may rewrite any ordered list to remove any load options that do not have corresponding load option variables. The boot manager can also, at its own discretion, provide an administrator with the ability to invoke manual maintenance operations as well. Examples include choosing the order of any or all load options, activating or deactivating load options, initiating OS-defined or platform-defined recovery, etc. In addition, if a platform intends to create *PlatformRecovery####*, before attempting to load and execute any *DriverOrder* or *BootOrder* entries, the firmware must create any and all *PlatformRecovery####* variables (See *Platform-Defined Boot Option Recovery*). The firmware should not, under normal operation, automatically remove any correctly formed *Boot####* variable currently referenced by the *BootOrder* or *BootNext* variables. Such removal should be limited to scenarios where the firmware is guided by direct user interaction.

The contents of *PlatformRecovery####* represent the final recovery options the firmware would have attempted had recovery been initiated during the current boot, and need not include entries to reflect contingencies such as significant hardware reconfiguration, or entries corresponding to specific hardware that the firmware is not yet aware of.

The behavior of the UEFI Boot Manager is impacted when Secure Boot is enabled, *Firmware/OS Key Exchange: Passing Public Keys*.

3.1.2 Load Option Processing

The boot manager is required to process the Driver load option entries before the Boot load option entries. If the *EFI_OS_INDICATIONS_START_OS_RECOVERY* bit has been set in *OsIndications*, the firmware shall attempt OS-defined recovery (See *OS-Defined Boot Option Recovery*) rather than normal boot processing. If the *EFI_OS_INDICATIONS_START_PLATFORM_RECOVERY* bit has been set in *OsIndications*, the firmware shall attempt platform-defined recovery (See *Platform-Defined Boot Option Recovery*) rather than normal boot processing or handling of the *EFI_OS_INDICATIONS_START_OS_RECOVERY* bit. In either case, both bits should be cleared.

Otherwise, the boot manager is also required to initiate a boot of the boot option specified by the *BootNext* variable as the first boot option on the next boot, and only on the next boot. The boot manager removes the *BootNext* variable before transferring control to the *BootNext* boot option. After the *BootNext* boot option is tried, the normal *BootOrder* list is used. To prevent loops, the boot manager deletes *BootNext* before transferring control to the preselected boot option.

If all entries of *BootNext* and *BootOrder* have been exhausted without success, or if the firmware has been instructed to attempt boot order recovery, the firmware must attempt boot option recovery (See *Boot Option Recovery*).

The boot manager must call *EFI_BOOT_SERVICES.LoadImage()* which supports at least *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* and *EFI_LOAD_FILE_PROTOCOL* for resolving load options. If *LoadImage()* succeeds, the boot manager must enable the watchdog timer for 5 minutes by using the *EFI_BOOT_SERVICES.SetWatchdogTimer()* boot service prior to calling *EFI_BOOT_SERVICES.StartImage()*. If a boot option returns control to the boot manager, the boot manager must disable the watchdog timer with an additional call to the *SetWatchdogTimer()* boot service.

If the boot image is not loaded via *EFI_BOOT_SERVICES.LoadImage()* the boot manager is required to check for a default application to boot. Searching for a default application to boot happens on both removable and fixed media types. This search occurs when the device path of the boot image listed in any boot option points directly to an *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* device and does not specify the exact file to load. The file discovery method is explained in *Boot Option Recovery*. The default media boot case of a protocol other than *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* is handled by the *EFI_LOAD_FILE_PROTOCOL* for the target device path and does not need to be handled by the boot manager.

The UEFI boot manager must support booting from a short-form device path that starts with the first element being a USB WWID (*USB WWID Device Path*) or a USB Class (*USB Class Device Path*) device path. For USB WWID, the boot manager must use the device vendor ID, device product id, and serial number, and must match any USB device in the system that contains this information. If more than one device matches the USB WWID device path, the boot manager will pick one arbitrarily. For USB Class, the boot manager must use the vendor ID, Product ID, Device Class, Device Subclass, and Device Protocol, and must match any USB device in the system that contains this information. If any of the ID, Product ID, Device Class, Device Subclass, or Device Protocol contain all F's (0xFFFF or 0xFF), this element is skipped for the purpose of matching. If more than one device matches the USB Class device path, the boot manager will pick one arbitrarily.

The boot manager must also support booting from a short-form device path that starts with the first element being a hard drive media device path (*Hard Drive Media Device Path*). The boot manager must use the GUID or signature and partition number in the hard drive device path to match it to a device in the system. If the drive supports the GPT partitioning scheme the GUID in the hard drive media device path is compared with the *UniquePartitionGuid* field of the GUID Partition Entry (*GPT Partition Entry*). If the drive supports the PC-AT MBR scheme the signature in the hard drive media device path is compared with the *UniqueMBRSignature* in the Legacy Master Boot Record (*Legacy MBR*). If a signature match is made, then the partition number must also be matched. The hard drive device path can be appended to the matching hardware device path and normal boot behavior can then be used. If more than one device matches the hard drive device path, the boot manager will pick one arbitrarily. Thus the operating system must ensure the uniqueness of the signatures on hard drives to guarantee deterministic boot behavior.

The boot manager must also support booting from a short-form device path that starts with the first element being a File Path Media Device Path (*File Path Media Device Path*). When the boot manager attempts to boot a short-form File Path Media Device Path, it will enumerate all removable media devices, followed by all fixed media devices, creating

boot options for each device. The boot option `FilePathList[0]` is constructed by appending short-form File Path Media Device Path to the device path of a media. The order within each group is undefined. These new boot options must not be saved to non volatile storage, and may not be added to *BootOrder*. The boot manager will then attempt to boot from each boot option. If a device does not support the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL*, but supports the *EFI_BLOCK_IO_PROTOCOL* protocol, then the EFI Boot Service *ConnectController* must be called for this device with *DriverImageHandle* and *RemainingDevicePath* set to `NULL` and the Recursive flag is set to *TRUE*. The firmware will then attempt to boot from any child handles produced using the algorithms outlined above.

The boot manager must also support booting from a short-form device path that starts with the first element being a URI Device Path (*URI Device Path*). When the boot manager attempts to boot a short-form URI Device Path, it could attempt to connect any device which will produce a device path protocol including a URI device path node until it matches a device, or fail to match any device. The boot manager will enumerate all *LoadFile* protocol instances, and invoke *LoadFile* protocol with *FilePath* set to the short-form device path during the matching process.

3.1.3 Load Options

Each load option variable contains an *EFI_LOAD_OPTION* descriptor that is a byte packed buffer of variable length fields.

```
typedef struct _EFI_LOAD_OPTION {
    UINT32                Attributes;
    UINT16                FilePathListLength;
    // CHAR16              Description[];
    // EFI_DEVICE_PATH_PROTOCOL  FilePathList[];
    // UINT8                OptionalData[];
} EFI_LOAD_OPTION;
```

Parameters

Attributes

The attributes for this load option entry. All unused bits must be zero and are reserved by the UEFI specification for future growth. See “Related Definitions.”

FilePathListLength

Length in bytes of the `FilePathList`. `OptionalData` starts at offset `sizeof(UINT32) + sizeof(UINT16) + StrSize(Description) + FilePathListLength` of the *EFI_LOAD_OPTION* descriptor.

Description

The user readable description for the load option. This field ends with a Null character.

FilePathList

A packed array of UEFI device paths. The first element of the array is a device path that describes the device and location of the Image for this load option. The `FilePathList[0]` is specific to the device type. Other device paths may optionally exist in the `FilePathList`, but their usage is OSV specific. Each element in the array is variable length, and ends at the device path end structure. Because the size of `Description` is arbitrary, this data structure is not guaranteed to be aligned on a natural boundary. This data structure may have to be copied to an aligned natural boundary before it is used.

OptionalData

The remaining bytes in the load option descriptor are a binary data buffer that is passed to the loaded image. If the field is zero bytes long, a `NULL` pointer is passed to the loaded image. The number of bytes in `OptionalData` can be computed by subtracting the starting offset of `OptionalData` from total size in bytes of the *EFI_LOAD_OPTION*.

Note

Each device path in the `FilePathList` can be a single instance or a multi-instance device path.

Related Definitions

```
// *****
// Attributes
// *****
#define LOAD_OPTION_ACTIVE                0x00000001
#define LOAD_OPTION_FORCE_RECONNECT      0x00000002
#define LOAD_OPTION_HIDDEN                0x00000008
#define LOAD_OPTION_CATEGORY              0x00001F00

#define LOAD_OPTION_CATEGORY_BOOT         0x00000000
#define LOAD_OPTION_CATEGORY_APP          0x00000100
// All values 0x00000200-0x00001F00 are reserved
```

Description

Calling *SetVariable()* creates a load option. The size of the load option is the same as the size of the *DataSize* argument to the *SetVariable()* call that created the variable. When creating a new load option, all undefined attribute bits must be written as zero. When updating a load option, all undefined attribute bits must be preserved.

If a load option is marked as *LOAD_OPTION_ACTIVE*, the boot manager will attempt to boot automatically using the device path information in the load option. This provides an easy way to disable or enable load options without needing to delete and re-add them.

If any *Driver####* load option is marked as *LOAD_OPTION_FORCE_RECONNECT*, then all of the UEFI drivers in the system will be disconnected and reconnected after the last *Driver####* load option is processed. This allows a UEFI driver loaded with a *Driver####* load option to override a UEFI driver that was loaded prior to the execution of the UEFI Boot Manager.

The executable indicated by *FilePathList[0]* in *Driver####* load option must be of type *EFI_IMAGE_SUBSYSTEM_EFI_BOOT_SERVICE_DRIVER* or *EFI_IMAGE_SUBSYSTEM_EFI_RUNTIME_DRIVER* otherwise the indicated executable will not be entered for initialization.

The executable indicated by *FilePathList[0]* in *SysPrep####*, *Boot####*, or *OsRecovery####* load option must be of type *EFI_IMAGE_SUBSYSTEM_EFI_APPLICATION*, otherwise the indicated executable will not be entered.

The *LOAD_OPTION_CATEGORY* is a sub-field of Attributes that provides details to the boot manager to describe how it should group the *Boot####* load options. This field is ignored for variables of the form *Driver####*, *SysPrep####*, or *OsRecovery####*.

Boot#### load options with *LOAD_OPTION_CATEGORY* set to *LOAD_OPTION_CATEGORY_BOOT* are meant to be part of the normal boot processing.

Boot#### load options with *LOAD_OPTION_CATEGORY* set to *LOAD_OPTION_CATEGORY_APP* are executables which are not part of the normal boot processing but can be optionally chosen for execution if boot menu is provided, or via Hot Keys. See *Launching Boot#### Load Options Using Hot Keys* for details.

Boot options with reserved category values, will be ignored by the boot manager.

If any *Boot####* load option is marked as *LOAD_OPTION_HIDDEN*, then the load option will not appear in the menu (if any) provided by the boot manager for load option selection.

3.1.4 Boot Manager Capabilities

The boot manager can report its capabilities through the global variable *BootOptionSupport*. If the global variable is not present, then an installer or application must act as if a value of 0 was returned.

```
#define EFI_BOOT_OPTION_SUPPORT_KEY      0x00000001
#define EFI_BOOT_OPTION_SUPPORT_APP      0x00000002
#define EFI_BOOT_OPTION_SUPPORT_SYSPREP  0x00000010
#define EFI_BOOT_OPTION_SUPPORT_COUNT    0x00000300
```

If *EFI_BOOT_OPTION_SUPPORT_KEY* is set then the boot manager supports launching of *Boot####* load options using key presses. If *EFI_BOOT_OPTION_SUPPORT_APP* is set then the boot manager supports boot options with *LOAD_OPTION_CATEGORY_APP*. If *EFI_BOOT_OPTION_SUPPORT_SYSPREP* is set then the boot manager supports boot options of form *SysPrep####*.

The value specified in *EFI_BOOT_OPTION_SUPPORT_COUNT* describes the maximum number of key presses which the boot manager supports in the *EFI_KEY_OPTION .KeyData.InputKeyCount*. This value is only valid if *EFI_BOOT_OPTION_SUPPORT_KEY* is set. Key sequences with more keys specified are ignored.

3.1.5 Launching Boot#### Applications

The boot manager may support a separate category of *Boot####* load option for applications. The boot manager indicates that it supports this separate category by setting the *EFI_BOOT_OPTION_SUPPORT_APP* in the *BootOptionSupport* global variable.

When an application's *Boot####* option is being added to the *BootOrder*, the installer should clear *LOAD_OPTION_ACTIVE* so that the boot manager does not attempt to automatically “boot” the application. If the boot manager indicates that it supports a separate application category, as described above, the installer should set *LOAD_OPTION_CATEGORY_APP*. If not, it should set *LOAD_OPTION_CATEGORY_BOOT*.

3.1.6 Launching Boot#### Load Options Using Hot Keys

The boot manager may support launching a *Boot####* load option using a special key press. If so, the boot manager reports this capability by setting *EFI_BOOT_OPTION_SUPPORT_KEY* in the *BootOptionSupport* global variable.

A boot manager which supports key press launch reads the current key information from the console. Then, if there was a key press, it compares the key returned against zero or more *Key####* global variables. If it finds a match, it verifies that the *Boot####* load option specified is valid and, if so, attempts to launch it immediately. The *####* in the *Key####* is a printable hexadecimal number ('0'-'9', 'A'-'F') with leading zeroes. The order which the *Key####* variables are checked is implementation-specific.

The boot manager may ignore *Key####* variables where the hot keys specified overlap with those used for internal boot manager functions. It is recommended that the boot manager delete these keys.

The *Key####* variables have the following format:

Prototype

```
typedef struct _EFI_KEY_OPTION {
    EFI_BOOT_KEY_DATA      KeyData;
    UINT32                 BootOptionCrc;
    UINT16                 BootOption;
    // EFI_INPUT_KEY        Keys[];
} EFI_KEY_OPTION;
```

Parameters

KeyData

Specifies options about how the key will be processed. Type `EFI_BOOT_KEY_DATA` is defined in “Related Definitions” below.

BootOptionCrc

The CRC-32 which should match the CRC-32 of the entire `EFI_LOAD_OPTION` to which `BootOption` refers. If the CRC-32s do not match this value, then this key option is ignored.

BootOption

The `Boot####` option which will be invoked if this key is pressed and the boot option is active (`LOAD_OPTION_ACTIVE` is set).

Keys

The key codes to compare against those returned by the `EFI_SIMPLE_TEXT_INPUT` and `EFI_SIMPLE_TEXT_INPUT_EX` protocols. The number of key codes (0-3) is specified by the `EFI_KEY_CODE_COUNT` field in `KeyOptions`.

Related Definitions

```
typedef union {
    struct {
        UINT32  Revision : 8;
        UINT32  ShiftPressed : 1;
        UINT32  ControlPressed : 1;
        UINT32  AltPressed : 1;
        UINT32  LogoPressed : 1;
        UINT32  MenuPressed : 1;
        UINT32  SysReqPressed : 1;
        UINT32  Reserved : 16;
        UINT32  InputKeyCount : 2;
    } Options;
    UINT32 PackedValue;
} EFI_BOOT_KEY_DATA;
```

Revision

Indicates the revision of the `EFI_KEY_OPTION` structure. This revision level should be 0.

ShiftPressed

Either the left or right Shift keys must be pressed (1) or must not be pressed (0).

ControlPressed

Either the left or right Control keys must be pressed (1) or must not be pressed (0).

AltPressed

Either the left or right Alt keys must be pressed (1) or must not be pressed (0).

LogoPressed

Either the left or right Logo keys must be pressed (1) or must not be pressed (0).

MenuPressed

The Menu key must be pressed (1) or must not be pressed (0).

SysReqPressed

The SysReq key must be pressed (1) or must not be pressed (0).

InputKeyCount

Specifies the actual number of entries in `EFI_KEY_OPTION`. Keys, from 0-3. If zero, then only the shift state is considered. If more than one, then the boot option will only be launched if all of the specified keys are pressed with the same shift state.

Example #1: ALT is the hot key. *KeyData.PackedValue* = 0x00000400.

Example #2: CTRL-ALT-P-R. *KeyData.PackedValue* = 0x80000600.

Example #3: CTRL-F1. *KeyData.PackedValue* = 0x40000200.

3.1.7 Required System Preparation Applications

A load option of the form *SysPrep####* is intended to designate a UEFI application that is required to execute in order to complete system preparation prior to processing of any *Boot####* variables. The execution order of *SysPrep####* applications is determined by the contents of the variable *SysPrepOrder* in a way directly analogous to the ordering of *Boot####* options by *BootOrder*.

The platform is required to examine all *SysPrep####* variables referenced in *SysPrepOrder*. If Attributes bit *LOAD_OPTION_ACTIVE* is set, and the application referenced by *FilePathList[0]* is present, the UEFI Applications thus identified must be loaded and launched in the order they appear in *SysPrepOrder* and prior to the launch of any load options of type *Boot####*.

When launched, the platform is required to provide the application loaded by *SysPrep####*, with the same services such as console and network as are normally provided at launch to applications referenced by a *Boot####* variable. *SysPrep####* application must exit and may not call *ExitBootServices()*. Processing of any Error Code returned at exit is according to system policy and does not necessarily change processing of following boot options. Any driver portion of the feature supported by *SysPrep####* boot option that is required to remain resident should be loaded by use of *Driver####* variable.

The Attributes option *LOAD_OPTION_FORCE_RECONNECT* is ignored for *SysPrep####* variables, and in the event that an application so launched performs some action that adds to the available hardware or drivers, the system preparation application shall itself utilize appropriate calls to *ConnectController()* or *DisconnectController()* to revise connections between drivers and hardware

After all *SysPrep####* variables have been launched and exited, the platform shall notify *EFI_EVENT_GROUP_READY_TO_BOOT* and *EFI_EVENT_GROUP_AFTER_READY_TO_BOOT* event groups. This should happen when the Boot Manager is about to load and execute *Boot####* variables with Attributes set to *LOAD_OPTION_CATEGORY_BOOT* according to the order defined by *BootOrder*.

3.2 Boot Manager Policy Protocol

3.2.1 EFI_BOOT_MANAGER_POLICY_PROTOCOL

Summary

This protocol is used by EFI Applications to request the UEFI Boot Manager to connect devices using platform policy.

GUID

```
#define EFI_BOOT_MANAGER_POLICY_PROTOCOL_GUID \
{ 0xFEDF8E0C, 0xE147, 0x11E3, \
  { 0x99, 0x03, 0xB8, 0xE8, 0x56, 0x2C, 0xBA, 0xFA } }
```

Protocol Interface Structure

```
typedef struct _EFI_BOOT_MANAGER_POLICY_PROTOCOL
EFI_BOOT_MANAGER_POLICY_PROTOCOL;
struct _EFI_BOOT_MANAGER_POLICY_PROTOCOL {
    UINT64 Revision;
    EFI_BOOT_MANAGER_POLICY_CONNECT_DEVICE_PATH ConnectDevicePath;
    EFI_BOOT_MANAGER_POLICY_CONNECT_DEVICE_CLASS ConnectDeviceClass;
};
```

ConnectDevicePath

Connect a Device Path following the platforms EFI Boot Manager policy.

ConnectDeviceClass

Connect a class of devices, named by EFI_GUID, following the platforms UEFI Boot Manager policy.

Description

The `EFI_BOOT_MANAGER_POLICY_PROTOCOL` is produced by the platform firmware to expose Boot Manager policy and platform specific `EFI_BOOT_SERVICES.ConnectController()` *EFI_BOOT_SERVICES.ConnectController()* behavior.

Related Definitions

```
#define EFI_BOOT_MANAGER_POLICY_PROTOCOL_REVISION 0x00010000
```

3.2.2 EFI_BOOT_MANAGER_POLICY_PROTOCOL.ConnectDevicePath()

Summary

Connect a device path following the platform's EFI Boot Manager policy.

```
Prototype
typedef
EFI_STATUS
(EFIAPI *EFI_BOOT_MANAGER_POLICY_CONNECT_DEVICE_PATH)(
    IN EFI_BOOT_MANAGER_POLICY_PROTOCOL *This,
    IN EFI_DEVICE_PATH *DevicePath,
    IN BOOLEAN Recursive
);
```

Parameters

This

A pointer to the `EFI_BOOT_MANAGER_POLICY_PROTOCOL` instance. Type `EFI_BOOT_MANAGER_POLICY_PROTOCOL` defined above.

DevicePath

Points to the start of the EFI device path to connect. If *DevicePath* is NULL then all the controllers in the system will be connected using the platform's EFI Boot Manager policy.

Recursive

If **TRUE**, then `ConnectController()` is called recursively until the entire tree of controllers below the controller specified by *DevicePath* have been created. If **FALSE**, then the tree of controllers is only expanded one level. If *DevicePath* is NULL then *Recursive* is ignored.

Description

The *ConnectDevicePath()* function allows the caller to connect a *DevicePath* using the same policy as the EFI Boot Manager.

If *Recursive* is **TRUE**, then *ConnectController()* is called recursively until the entire tree of controllers below the controller specified by *DevicePath* have been created. If *Recursive* is **FALSE**, then the tree of controllers is only expanded one level. If *DevicePath* is NULL then *Recursive* is ignored.

Status Codes Returned

EFI_SUCCESS	The <i>DevicePath</i> was connected
EFI_NOT_FOUND	The <i>DevicePath</i> was not found
EFI_NOT_FOUND	No driver was connected to <i>DevicePath</i> .
EFI_SECURITY_VIOLATION	The user has no permission to start UEFI device drivers
EFI_UNSUPPORTED	The current TPL is not <i>TPL_APPLICATION</i> .

3.2.3 EFI_BOOT_MANAGER_POLICY_PROTOCOL.ConnectDeviceClass()

Summary

Connect a class of devices using the platform Boot Manager policy.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BOOT_MANAGER_POLICY_CONNECT_DEVICE_CLASS)(
    IN EFI_BOOT_MANAGER_POLICY_PROTOCOL    *This,
    IN EFI_GUID                            *Class
);
```

Parameters

This

A pointer to the `EFI_BOOT_MANAGER_POLICY_PROTOCOL` instance. Type `EFI_BOOT_MANAGER_POLICY_PROTOCOL` is defined above.

Class

A pointer to an *EFI_GUID* that represents a class of devices that will be connected using the Boot Manager's platform policy.

Description

The *ConnectDeviceClass()* function allows the caller to request that the Boot Manager connect a class of devices.

If *Class* is *EFI_BOOT_MANAGER_POLICY_CONSOLE_GUID* then the Boot Manager will use platform policy to connect consoles. Some platforms may restrict the number of consoles connected as they attempt to fast boot, and calling *ConnectDeviceClass()* with a *Class* value of *EFI_BOOT_MANAGER_POLICY_CONSOLE_GUID* must connect the set of consoles that follow the Boot Manager platform policy, and the *EFI_SIMPLE_TEXT_INPUT_PROTOCOL*, *EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL*, and the *EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL* are produced on the connected handles. The Boot Manager may restrict which consoles get connect due to platform policy, for example a security policy may require that a given console is not connected.

If *Class* is *EFI_BOOT_MANAGER_POLICY_NETWORK_GUID* then the Boot Manager will connect the protocols the platform supports for UEFI general purpose network applications on one or more handles. The protocols associated with UEFI general purpose network applications are defined in *Platform-Specific Elements*, list item number 7. If more than one network controller is available a platform will connect, one, many, or all of the networks based on platform policy. Connecting UEFI networking protocols, like *EFI_DHCP4_PROTOCOL*, does not establish connections on

the network. The UEFI general purpose network application that called *ConnectDeviceClass()* may need to use the published protocols to establish the network connection. The Boot Manager can optionally have a policy to establish a network connection.

If *Class* is *EFI_BOOT_MANAGER_POLICY_STORAGE_GUID* then the Boot Manager will connect the protocols associated with the discoverable storage devices. These protocols may include *EFI_BLOCK_IO_PROTOCOL*, *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL*, or other storage protocols relevant to the device. The platform may choose to restrict the connected devices based on its policy. For example, the platform may exclude external peripherals or only include specific, well-known devices.

If *Class* is *EFI_BOOT_MANAGER_POLICY_CONNECT_ALL_GUID* then the Boot Manager will connect all UEFI drivers using the UEFI Boot Service *EFI_BOOT_SERVICES.ConnectController()*. If the Boot Manager has policy associated with connect all UEFI drivers this policy will be used.

A platform can also define platform specific *Class* values as a properly generated *EFI_GUID* would never conflict with this specification.

Related Definitions

```
#define EFI_BOOT_MANAGER_POLICY_CONSOLE_GUID \
    { 0xCAB0E94C, 0xE15F, 0x11E3, \
      0x91, 0x8D, 0xB8, 0xE8, 0x56, 0x2C, 0xBA, 0xFA } }
#define EFI_BOOT_MANAGER_POLICY_NETWORK_GUID \
    { 0xD04159DC, 0xE15F, 0x11E3, \
      0xB2, 0x61, 0xB8, 0xE8, 0x56, 0x2C, 0xBA, 0xFA } }
#define EFI_BOOT_MANAGER_POLICY_STORAGE_GUID \
    { 0xCD68FE79, 0xD3CB, 0x436E, \
      0xA8, 0x50, 0xF4, 0x43, 0xC8, 0x8C, 0xFB, 0x49 } }
#define EFI_BOOT_MANAGER_POLICY_CONNECT_ALL_GUID \
    { 0x113B2126, 0xFC8A, 0x11E3, \
      0xBD, 0x6C, 0xB8, 0xE8, 0x56, 0x2C, 0xBA, 0xFA } }
```

Status Codes Returned

EFI_SUCCESS	At least one devices of the <i>Class</i> was connected.
EFI_DE_ERROR	Devices were not connected due to an error.
EFI_NOT_FOUND	The <i>Class</i> is not supported by the platform.
EFI_UNSUPPORTED	The current TPL is not <i>TPL_APPLICATION</i> .

3.3 Globally Defined Variables

This section defines a set of variables that have architecturally defined meanings. In addition to the defined data content, each such variable has an architecturally defined attribute that indicates when the data variable may be accessed. The variables with an attribute of NV are nonvolatile. This means that their values are persistent across resets and power cycles. The value of any environment variable that does not have this attribute will be lost when power is removed from the system and the state of firmware reserved memory is not otherwise preserved. The variables with an attribute of BS are only available before *EFI_BOOT_SERVICES.ExitBootServices()* is called. This means that these environment variables can only be retrieved or modified in the preboot environment. They are not visible to an operating system. Environment variables with an attribute of RT are available before and after *ExitBootServices()* is called. Environment variables of this type can be retrieved and modified in the preboot environment, and from an operating system. The variables with an attribute of AT are variables with a time-based authenticated write access defined in *Using the EFI_VARIABLE_AUTHENTICATION_3 descriptor*. All architecturally defined variables use the *EFI_GLOBAL_VARIABLE_VendorGuid*.

```
#define EFI_GLOBAL_VARIABLE \
{0x8BE4DF61,0x93CA,0x11d2,\
 {0xAA,0x0D,0x00,0xE0,0x98,0x03,0x2B,0x8C}}
```

To prevent name collisions with possible future globally defined variables, other internal firmware data variables that are not defined here must be saved with a unique *VendorGuid* other than *EFI_GLOBAL_VARIABLE* or any other GUID defined by the UEFI Specification. Implementations must only permit the creation of variables with a UEFI Specification-defined *VendorGuid* when these variables are documented in the UEFI Specification.

Table 3.3: Global Variables

Variable Name	Attribute	Description
AuditMode	BS, RT	Whether the system is operating in Audit Mode (1) or not (0). All other values are reserved. Should be treated as read-only except when DeployedMode is 0. Always becomes read-only after ExitBootServices() is called.
Boot####	NV, BS, RT	A boot load option. #### is a printed hex value. No 0x or h is included in the hex value.
BootCurrent	BS, RT	The boot option that was selected for the current boot.
BootNext	NV, BS, RT	The boot option for the next boot only.
BootOrder	NV, BS, RT	The ordered boot option load list.
BootOptionSupport	BS,RT,	The types of boot options supported by the boot manager. Should be treated as read-only.
ConIn	NV, BS, RT	The device path of the default input console.
ConInDev	BS, RT	The device path of all possible console input devices.
ConOut	NV, BS, RT	The device path of the default output console.
ConOutDev	BS, RT	The device path of all possible console output devices.
dbDefault	BS, RT	The OEM's default secure boot signature store. Should be treated as read-only.
dbrDefault	BS, RT	The OEM's default OS Recovery signature store. Should be treated as read-only.
dbtDefault	BS, RT	The OEM's default secure boot timestamp signature store. Should be treated as read-only.
dbxDefault	BS, RT	The OEM's default secure boot blacklist signature store. Should be treated as read-only.
DeployedMode	BS, RT	Whether the system is operating in Deployed Mode (1) or not (0). All other values are reserved. Should be treated as read-only when its value is 1. Always becomes read-only after ExitBootServices() is called.
devAuthBoot	BS, RT	Whether the platform firmware is operating in device authentication boot mode (1) or not (0). All other values are reserved. Should be treated as read-only.
devdbDefault	BS, RT	The OEM's default device authentication signature store. Should be treated as read-only.
Driver####	NV, BS, RT	A driver load option. #### is a printed hex value.
DriverOrder	NV, BS, RT	The ordered driver load option list.
ErrOut	NV, BS, RT	The device path of the default error output device.
ErrOutDev	BS, RT	The device path of all possible error output devices.
HwErrRecSupport	NV, BS, RT	Identifies the level of hardware error record persistence support implemented by the platform. This variable is only modified by firmware and is read-only to the OS.
KEK	NV, BS, RT,AT	The Key Exchange Key Signature Database.

continues on next page

Table 3.3 – continued from previous page

KEKDefault	BS, RT	The OEM's default Key Exchange Key Signature Database. Should be treated as read-only.
Key####	NV, BS, RT	Describes hot key relationship with a Boot#### load option.
OsIndications	NV, BS, RT	Allows the OS to request the firmware to enable certain features and to take certain actions.
OsIndicationsSupported	BS, RT	Allows the firmware to indicate supported features and actions to the OS.
OsRecoveryOrder	BS,RT,NV,AT	OS-specified recovery options.
PK	NV, BS, RT,AT	The public Platform Key.
PKDefault	BS, RT	The OEM's default public Platform Key. Should be treated as read-only.
PlatformLangCodes	BS, RT	The language codes that the firmware supports.
PlatformLang	NV, BS, RT	The language code that the system is configured for.
PlatformRecovery####	BS, RT	Platform-specified recovery options. These variables are only modified by firmware and are read-only to the OS.
SignatureSupport	BS, RT	Array of GUIDs representing the type of signatures supported by the platform firmware. Should be treated as read-only.
SecureBoot	BS, RT	Whether the platform firmware is operating in Secure boot mode (1) or not (0). All other values are reserved. Should be treated as read-only.
SetupMode	BS, RT	Whether the system should require authentication on SetVariable() requests to Secure Boot policy variables (0) or not (1). Should be treated as read-only. The system is in "Setup Mode" when SetupMode==1, AuditMode==0, and DeployedMode==0.
SysPrep####	NV, BS, RT	A System Prep application load option containing an <i>EFI_LOAD_OPTION</i> descriptor. #### is a printed hex value.
SysPrepOrder	NV, BS, RT	The ordered System Prep Application load option list.
Timeout	NV, BS, RT	The firmware's boot managers timeout, in seconds, before initiating the default boot selection.
VendorKeys	BS, RT	Whether the system is configured to use only vendor-provided keys or not. Should be treated as read-only.

The *PlatformLangCodes* variable contains a null-terminated ASCII string representing the language codes that the firmware can support. At initialization time the firmware computes the supported languages and creates this data variable. Since the firmware creates this value on each initialization, its contents are not stored in nonvolatile memory. This value is considered read-only. *PlatformLangCodes* is specified in Native RFC 4646 format. See [Formats — Language Codes and Language Code Arrays](#).

The *PlatformLang* variable contains a null-terminated ASCII string language code that the machine has been configured for. This value may be changed to any value supported by *PlatformLangCodes*. If this change is made in the preboot environment, then the change will take effect immediately. If this change is made at OS runtime, then the change does not take effect until the next boot. If the language code is set to an unsupported value, the firmware will choose a supported default at initialization and set *PlatformLang* to a supported value. *PlatformLang* is specified in Native RFC 4646 array format. See [Formats — Language Codes and Language Code Arrays](#).

The *Timeout* variable contains a binary *UINT16* that supplies the number of seconds that the firmware will wait before initiating the original default boot selection. A value of 0 indicates that the default boot selection is to be initiated immediately on boot. If the value is not present, or contains the value of 0xFFFF then firmware will wait for user input before booting. This means the default boot selection is not automatically started by the firmware.

The *ConIn*, *ConOut*, and *ErrOut* variables each contain an *EFI Device Path Protocol* descriptor that defines the default device to use on boot. Changes to these values made in the preboot environment take effect immediately. Changes to these values at OS runtime do not take effect until the next boot. If the firmware cannot resolve the device path, it is allowed to automatically replace the values, as needed, to provide a console for the system. If the device path starts

with a USB Class device path (*USB Class Device Path*), then any input or output device that matches the device path must be used as a console if it is supported by the firmware.

The *ConInDev* , *ConOutDev* , and *ErrOutDev* variables each contain an *EFI_DEVICE_PATH_PROTOCOL* descriptor that defines all the possible default devices to use on boot. These variables are volatile, and are set dynamically on every boot. *ConIn* , *ConOut* , and *ErrOut* are always proper subsets of *ConInDev* , *ConOutDev* , and *ErrOutDev* .

Each *Boot####* variable contains an *EFI_LOAD_OPTION*. Each *Boot####* variable is the name “Boot” appended with a unique four digit hexadecimal number. For example, *Boot0001*, *Boot0002*, *Boot0A02*, etc.

The *OsRecoveryOrder* variable contains an array of *EFI_GUID* structures. Each *EFI_GUID* structure specifies a namespace for variables containing OS-defined recovery entries (See *OS-Defined Boot Option Recovery*). Write access to this variable is controlled by the security key database dbr (*Using the EFI_VARIABLE_AUTHENTICATION_3 descriptor*).

PlatformRecovery#### variables share the same structure as *Boot####* variables. These variables are processed when the system is performing recovery of boot options.

The *BootOrder* variable contains an array of *UINT16* ’s that make up an ordered list of the *Boot####* options. The first element in the array is the value for the first logical boot option, the second element is the value for the second logical boot option, etc. The *BootOrder* order list is used by the firmware’s boot manager as the default boot order.

The *BootNext* variable is a single *UINT16* that defines the *Boot####* option that is to be tried first on the next boot. After the *BootNext* boot option is tried the normal *BootOrder* list is used. To prevent loops, the boot manager deletes this variable before transferring control to the preselected boot option.

The *BootCurrent* variable is a single *UINT16* that defines the *Boot####* option that was selected on the current boot. The platform sets this variable before signaling *EFI_EVENT_GROUP_READY_TO_BOOT*. This variable is not set when attempting to launch *OsRecovery###* or *PlatformRecovery###* options.

The *BootOptionSupport* variable is a *UINT32* that defines the types of boot options supported by the boot manager.

Each *Driver####* variable contains an *EFI_LOAD_OPTION*. Each load option variable is appended with a unique number, for example *Driver0001*, *Driver0002*, etc.

The *DriverOrder* variable contains an array of *UINT16* ’s that make up an ordered list of the *Driver####* variable. The first element in the array is the value for the first logical driver load option, the second element is the value for the second logical driver load option, etc. The *DriverOrder* list is used by the firmware’s boot manager as the default load order for UEFI drivers that it should explicitly load.

The *Key####* variable associates a key press with a single boot option. Each *Key####* variable is the name “Key” appended with a unique four digit hexadecimal number. For example, *Key0001*, *Key0002*, *Key00A0*, etc.

The *HwErrRecSupport* variable contains a binary *UINT16* that supplies the level of support for Hardware Error Record Persistence (*Hardware Error Record Persistence*) that is implemented by the platform. If the value is not present, then the platform implements no support for Hardware Error Record Persistence. A value of zero indicates that the platform implements no support for Hardware Error Record Persistence. A value of 1 indicates that the platform implements Hardware Error Record Persistence as defined in *Hardware Error Record Persistence*. Firmware initializes this variable. All other values are reserved for future use.

The *SetupMode* variable is an 8-bit unsigned integer that defines whether the system is should require authentication (0) or not (1) on *SetVariable()* requests to Secure Boot Policy Variables. Secure Boot Policy Variables include:

- The global variables *PK* , *KEK* , and *OsRecoveryOrder*
- All variables named *OsRecovery####* under all VendorGuids
- All variables with the VendorGuid *EFI_IMAGE_SECURITY_DATABASE_GUID*.

Secure Boot Policy Variables must be created using the *EFI_VARIABLE_AUTHENTICATION_2* structure.

The *AuditMode* variable is an 8-bit unsigned integer that defines whether the system is currently operating in Audit Mode.

The *DeployedMode* variable is an 8-bit unsigned integer that defines whether the system is currently operating in Deployed Mode.

The *KEK* variable contains the current Key Exchange Key database.

The *PK* variable contains the current Platform Key.

The *VendorKeys* variable is an 8-bit unsigned integer that defines whether the Security Boot Policy Variables have been modified by anyone other than the platform vendor or a holder of the vendor-provided keys. A value of 0 indicates that someone other than the platform vendor or a holder of the vendor-provided keys has modified the Secure Boot Policy Variables. Otherwise, the value will be 1.

The *KEKDefault* variable, if present, contains the platform-defined Key Exchange Key database. This is not used at runtime but is provided in order to allow the OS to recover the OEM's default key setup. The contents of this variable do not include an *EFI_VARIABLE_AUTHENTICATION* or *EFI_VARIABLE_AUTHENTICATION2* structure.

The *PKDefault* variable, if present, contains the platform-defined Platform Key. This is not used at runtime but is provided in order to allow the OS to recover the OEM's default key setup. The contents of this variable do not include an *EFI_VARIABLE_AUTHENTICATION2* structure.

The *dbDefault* variable, if present, contains the platform-defined secure boot signature database. This is not used at runtime but is provided in order to allow the OS to recover the OEM's default key setup. The contents of this variable do not include an *EFI_VARIABLE_AUTHENTICATION2* structure.

The *dbrDefault* variable, if present, contains the platform-defined secure boot authorized recovery signature database. This is not used at runtime but is provided in order to allow the OS to recover the OEM's default key setup. The contents of this variable do not include an *EFI_VARIABLE_AUTHENTICATION2* structure.

The *dbtDefault* variable, if present, contains the platform-defined secure boot timestamp signature database. This is not used at runtime but is provided in order to allow the OS to recover the OEM's default key setup. The contents of this variable do not include an *EFI_VARIABLE_AUTHENTICATION2* structure.

The *dbxDefault* variable, if present, contains the platform-defined secure boot blacklist signature database. This is not used at runtime but is provided in order to allow the OS to recover the OEM's default key setup. The contents of this variable do not include an *EFI_VARIABLE_AUTHENTICATION2* structure.

The *SignatureSupport* variable returns an array of GUIDs, with each GUID representing a type of signature which the platform firmware supports for images and other data. The different signature types are described in "Signature Database".

The *SecureBoot* variable is an 8-bit unsigned integer that defines whether the platform firmware is operating with Secure Boot enabled. A value of 1 indicates that platform firmware performs driver and boot application signature verification as specified in *UEFI Image Validation* during the current boot. A value of 0 indicates that driver and boot application signature verification is not active during the current boot. The SecureBoot variable is initialized prior to Secure Boot image authentication and thereafter should be treated as read-only and immutable. Its initialization value is determined by platform policy but must be 0 if the platform is in Setup Mode or Audit Mode during its initialization.

The *OsIndicationsSupported* variable indicates which of the OS indication features and actions that the firmware supports. This variable is recreated by firmware every boot, and cannot be modified by the OS (see *SetVariable()* Attributes usage rules once *ExitBootServices()* is performed).

The *OsIndications* variable is used to indicate which features the OS wants firmware to enable or which actions the OS wants the firmware to take. The OS will supply this data with a *SetVariable()* call. *Exchanging information between the OS and Firmware* for the variable definition.

The *devAuthBoot* variable is an 8-bit unsigned integer that defines whether the platform firmware is operating with device authentication boot enabled. A value of 1 indicates that platform firmware performs device authentication as specified in Device Authentication during the current boot. A value of 0 indicates that device authentication is not active during the current boot. The *devAuthBoot* variable is initialized prior to device authentication and thereafter should be treated as read-only and immutable. Its initialization value is determined by platform policy.

The `devdbDefault` variable, if present, contains the platform-defined device authentication signature database. This is not used at runtime but is provided in order to allow the OS to recover the OEM's default key setup. The contents of this variable do not include an `EFI_VARIABLE_AUTHENTICATION2` structure.

3.4 Boot Option Recovery

Boot option recovery consists of two independent parts, operating system-defined recovery and platform-defined recovery. OS-defined recovery is an attempt to allow installed operating systems to recover any needed boot options, or to launch full operating system recovery. Platform-defined recovery includes any remedial actions performed by the platform as a last resort when no operating system is found, such as the Default Boot Behavior (*Boot Option Variables Default Boot Behavior*). This could include behaviors such as warranty service reconfiguration or diagnostic options.

In the event that boot option recovery must be performed, the boot manager must first attempt OS-defined recovery, re-attempt normal booting via `Boot####` and `BootOrder` variables, and finally attempt platform-defined recovery if no options have succeeded.

3.4.1 OS-Defined Boot Option Recovery

If the `EFI_OS_INDICATIONS_START_OS_RECOVERY` bit is set in `OsIndications` , or if processing of `BootOrder` does not result in success, the platform must process OS-defined recovery options. In the case where OS-defined recovery is entered due to `OsIndications` , `SysPrepOrder` and `SysPrep####` variables should not be processed. Note that in order to avoid ambiguity in intent, this bit is ignored in `OsIndications` if `EFI_OS_INDICATIONS_START_PLATFORM_RECOVERY` is set.

OS-defined recovery uses the `OsRecoveryOrder` variable, as well as variables created with vendor specific `VendorGuid` values and a name following the pattern `OsRecovery####`. Each of these variables must be an authenticated variable with the `EFI_VARIABLE_TIME_BASED_AUTHENTICATED_WRITE_ACCESS` attribute set.

To process these variables, the boot manager iterates over the array of `EFI_GUID` structures in the `OsRecoveryOrder` variable, and each GUID specified is treated as a `VendorGuid` associated with a series of variable names. For each GUID, the firmware attempts to load and execute, in hexadecimal sort order, every variable with that GUID and a name following the pattern `OsRecovery####`. These variables have the same format as `Boot####` variables, and the boot manager must verify that each variable it attempts to load was created with a public key that is associated with a certificate chaining to one listed in the authorized recovery signature database `dbr` and not in the forbidden signature database, or is created by a key in the Key Exchange Key database `KEK` or the current Platform Key `PK` .

If the boot manager finishes processing `OsRecovery####` options without `EFI_BOOT_SERVICES.ExitBootServices()` or `ResetSystem()` having been called, it must attempt to process `BootOrder` a second time. If booting does not succeed during that process, OS-defined recovery has failed, and the boot manager must attempt platform-based recovery.

If, while processing `OsRecovery####` variables, the boot manager encounters an entry which cannot be loaded or executed due to a security policy violation, it must ignore that variable.

3.4.2 Platform-Defined Boot Option Recovery

If the `EFI_OS_INDICATIONS_START_PLATFORM_RECOVERY` bit is set in `OsIndications` , or if OS-defined recovery has failed, the system firmware must commence with platform-specific recovery by iterating its `PlatformRecovery####` variables in the same manner as `OsRecovery####` , but must stop processing if any entry is successful. In the case where platform-specific recovery is entered due to `OsIndications` , `SysPrepOrder` and `SysPrep####` variables should not be processed.

3.4.3 Boot Option Variables Default Boot Behavior

The default state of globally-defined variables is firmware vendor specific. However the boot options require a standard default behavior in the exceptional case that valid boot options are not present on a platform. The default behavior must be invoked any time the *BootOrder* variable does not exist or only points to nonexistent boot options, or if no entry in *BootOrder* can successfully be executed.

If system firmware supports boot option recovery as described in *Boot Option Recovery*, system firmware must include a *PlatformRecovery####* variable specifying a short-form File Path Media Device Path (*Load Option Processing*) containing the platform default file path for removable media (*UEFI Image Types*). It is recommended for maximal compatibility with prior versions of this specification that this entry be the first such variable, though it may be at any position within the list.

It is expected that this default boot will load an operating system or a maintenance utility. If this is an operating system setup program it is then responsible for setting the requisite environment variables for subsequent boots. The platform firmware may also decide to recover or set to a known set of boot options.

3.5 Boot Mechanisms

EFI can boot from a device using the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* or the *EFI_LOAD_FILE_PROTOCOL*. A device that supports the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* must materialize a file system protocol for that device to be bootable. If a device does not wish to support a complete file system it may produce an *EFI_LOAD_FILE_PROTOCOL* which allows it to materialize an image directly. The Boot Manager will attempt to boot using the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* first. If that fails, then the *EFI_LOAD_FILE_PROTOCOL* will be used.

3.5.1 Boot via the Simple File Protocol

When booting via the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL*, the *FilePath* will start with a device path that points to the device that implements the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* or the *EFI_BLOCK_IO_PROTOCOL*. The next part of the *FilePath* may point to the file name, including subdirectories, which contain the bootable image. If the file name is a null device path, the file name must be generated from the rules defined below.

If the *FilePathList[0]* device does not support the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL*, but supports the *EFI_BLOCK_IO_PROTOCOL* protocol, then the EFI Boot Service *EFI_BOOT_SERVICES.ConnectController()* must be called for *FilePathList[0]* with *DriverImageHandle* and *RemainingDevicePath* set to *NULL* and the *Recursive* flag is set to *TRUE*. The firmware will then attempt to boot from any child handles produced using the algorithms outlined below.

The format of the file system specified is contained in *File System Format*. While the firmware must produce an *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* that understands the UEFI file system, any file system can be abstracted with the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* interface.

3.5.1.1 Removable Media Boot Behavior

To generate a file name when none is present in the *FilePath*, the firmware must append a default file name in the form `\EFI\BOOT\BOOT{machine type short-name}.EFI` where machine type short-name defines a PE32+ image format architecture. Each file only contains one UEFI image type, and a system may support booting from one or more images types. See Table 3.4 for a list of the UEFI image types.

Table 3.4: UEFI Image Types

	File Name Convention	PE Executable Machine Type *
32-bit	BOOTIA32.EFI	0x14c
x64	BOOTx64.EFI	0x8664
Itanium architecture	BOOTIA64.EFI	0x200
AArch32 architecture	BOOTARM.EFI	0x01c2
AArch64 architecture	BOOTAA64.EFI	0xAA64
RISC-V 32-bit architecture	BOOTRISCV32.EFI	0x5032
RISC-V 64-bit architecture	BOOTRISCV64.EFI	0x5064
RISC-V 128-bit architecture	BOOTRISCV128.EFI	0x5128
LoongArch32 architecture	BOOTLOONGARCH32.EFI	0x6232
LoongArch64 architecture	BOOTLOONGARCH64.EFI	0x6264

Note

The PE Executable machine type is contained in the machine field of the COFF file header as defined in the Microsoft Portable Executable and Common Object File Format Specification, Revision 6.0*

Media may support multiple architectures by simply having a file `\EFI\BOOT\BOOT{machine type short-name}.EFI` for each possible machine type.

3.5.2 Boot via the Load File Protocol

When booting via the *EFI_LOAD_FILE_PROTOCOL* protocol, the *FilePath* is a device path that points to a device that “speaks” the *EFI_LOAD_FILE_PROTOCOL*. The image is loaded directly from the device that supports the *EFI_LOAD_FILE_PROTOCOL*. The remainder of the *FilePath* will contain information that is specific to the device. Firmware passes this device-specific data to the loaded image, but does not use it to load the image. If the remainder of the *FilePath* is a null device path it is the loaded image’s responsibility to implement a policy to find the correct boot device.

The *EFI_LOAD_FILE_PROTOCOL* is used for devices that do not directly support file systems. Network devices commonly boot in this model where the image is materialized without the need of a file system.

3.5.2.1 Network Booting

Network booting is described by the *Preboot eXecution Environment (PXE) BIOS Support Specification*. PXE specifies UDP, DHCP, and TFTP network protocols that a booting platform can use to interact with an intelligent system load server. UEFI defines special interfaces that are used to implement PXE. These interfaces are contained in the *EFI_PXE_BASE_CODE_PROTOCOL*.

3.5.2.2 Future Boot Media

Since UEFI defines an abstraction between the platform and the OS and its loader it should be possible to add new types of boot media as technology evolves. The OS loader will not necessarily have to change to support new types of boot. The implementation of the UEFI platform services may change, but the interface will remain constant. The OS will require a driver to support the new type of boot media so that it can make the transition from UEFI boot services to OS control of the boot media.